

03 — JSON Parsing & Data Normalization

What it is

JSON parsing is reading a structured data file (JSON = JavaScript Object Notation) into Python objects you can work with — dictionaries, lists, strings, numbers. **Data normalization** is the cleanup step where you fix inconsistencies in the raw data so that the rest of your pipeline can trust a single, predictable format.

In our project, the source data arrives as JSON exports from a Hungarian B2B platform's database. The data looks clean at first glance, but it has hidden inconsistencies that would break downstream code if we don't fix them at the very first step.

Real-life analogy

Imagine you're receiving inventory shipments from three different warehouses. Warehouse A labels box weights as "5 kg" (a text label). Warehouse B writes 5 (a plain number). Warehouse C sends "[5, 10, 15]" as a text string instead of an actual list of numbers. They all mean the same thing, but your automated sorting machine only understands one format.

Normalization is having one person at the loading dock who opens every box, reads the label, and re-stamps it in the one format your system expects — before anything goes further down the conveyor belt. If you skip this step, the sorting machine jams randomly when it hits Warehouse C's weird text-formatted lists, and the bug is incredibly hard to trace back to the source.

Why we used it here

- **R1** requires ingesting ~3,000 categories and ~471 companies from `data/*.json` files.
- The dataset has real-world messiness documented in REQUIREMENTS.md §4:
- List fields like `brands` arrive as `'["ABS", "ABS MF"]'` (a string that *looks like* a list) in some records, but as an actual JSON array `["ABS", "ABS MF"]` in others — the *same type of data* encoded two different ways.
- Numeric fields like `nr_employees` arrive as `"13"` (a string) instead of `13` (a number).
- `description` is null or empty in 34% of company records.
- `address` is empty in 100% of company records.

If we don't fix these at the extraction boundary, every downstream module (narrative builder, embedding generator, search API) would need its own defensive checks — duplicating logic and creating a minefield of subtle bugs.

File: `bilingual_etl/extract/source.py`

Code walkthrough

```
# The key insight: normalize at the boundary, not downstream.
# Everything after source.py can trust that lists are lists and numbers are numbers.
```

```
STRINGIFIED_LIST_FIELDS = {"brands", "certificates", "services", "counties"}
# These specific fields arrive as strings like '["ABS", "ABS MF"]' in the
# company data. We know this from our Phase 0 dataset analysis.
```

```
NUMERIC_FIELDS = {"nr_employees", "year_founded", "registered_capital",
                  "profit_after_tax", "price_in"}
# These arrive as strings like "13" or "1,000,000" – need casting to int/float.
```

```
def _safe_parse_list(value):
    """Convert any input into a proper Python list.

    Handles four cases we found in the real data:
    None           → []
    ["a", "b"]     → ["a", "b"]     (already a list, pass through)
    '["a", "b"]'   → ["a", "b"]     (stringified JSON, parse it)
    "single"       → ["single"]     (bare string, wrap in list)
    """
    if value is None:
        return []
    if isinstance(value, list):
        return value          # Already the right type, nothing to do
    if isinstance(value, str):
        stripped = value.strip()
        if not stripped or stripped == "[]":
            return []         # Empty string or empty brackets
        try:
            parsed = json.loads(stripped)  # Try parsing as JSON
            if isinstance(parsed, list):
                return parsed              # Successfully parsed a stringified list
        except (json.JSONDecodeError, TypeError):
            pass
        return [stripped]              # Not JSON – treat as a single-item list
    return [value]                    # Anything else, wrap in list


def _safe_parse_number(value):
    """Convert string numbers to int/float, handling edge cases.

    "42"           → 42             (string to int)
    "3.14"         → 3.14           (string to float)
    "1,000,000"    → 1000000       (strip commas first)
    ""            → None           (empty = no data)
    "-"           → None           (dash = no data, common in Hungarian datasets)
    None          → None
    """
    if value is None:
        return None
    if isinstance(value, (int, float)):
        return value          # Already a number
    if isinstance(value, str):
        stripped = value.strip().replace(",", "").replace(" ", "")
        if not stripped or stripped == "-":
            return None        # No data
        try:
            return int(stripped)
        except ValueError:
            try:
                return float(stripped)
            except ValueError:
                return None     # Can't parse – treat as missing
    return None


def _normalize_company(raw: dict) -> dict:
```

```
"""Apply all normalizations to a raw company record."""
company = dict(raw)    # Shallow copy – don't mutate the source

# Fix the inconsistent list encoding
for field in STRINGIFIED_LIST_FIELDS:
    if field in company:
        company[field] = _safe_parse_list(company[field])

# Cast string numbers to actual numbers
for field in NUMERIC_FIELDS:
    if field in company:
        company[field] = _safe_parse_number(company[field])

# Ensure description/address are always strings (never None)
# so downstream code doesn't need null checks
if not company.get("description"):
    company["description"] = ""
if not company.get("address"):
    company["address"] = ""

return company
```

How to explain this in an interview / to a teammate

"We normalize all source data at the extraction boundary — the very first step of the ETL pipeline — so that every downstream module can trust a consistent format. The raw JSON files had three kinds of inconsistency: list fields encoded sometimes as native arrays and sometimes as stringified JSON, numeric fields stored as strings, and widespread nulls in description and address fields. Rather than scattering defensive parsing across the whole codebase, we wrote two utility functions — `_safe_parse_list` and `_safe_parse_number` — that handle every variant we found in the real data, and we apply them once during extraction. This is a common ETL pattern: validate and normalize at ingestion, trust internally."